Edge Agentic Assistant (EAA)

Complete Backend Low-Level System Design (LLD) & AI CodeGen Architecture Blueprint

This document establishes the exhaustive, machine-executable Low-Level System Design (LLD) for the Edge Agentic Assistant (EAA) backend and edge subsystems. It details the precise folder structure, component-level library stack, interface contracts (gRPC, ZeroMQ, Pydantic, OpenAMP C-structs), linkage topologies, and concrete object-oriented design specifications (classes, attributes, methods, types, state matrices, and invariant rules) required for an AI coding agent or engineering team to generate production software without ambiguity.

1. Unified Monorepo Directory & File Structure

The repository is organized into isolated microservice containers, shared type contracts, and MCU embedded C++ firmware:

```
eaa-monorepo/
├── docker-compose.yml
├── shared/                              # Shared contracts across container boundaries
│   ├── proto/
│   │   ├── telemetry.proto              # gRPC Telemetry stream definition
│   │   ├── safety.proto                 # gRPC Safety validation RPCs
│   │   └── tool_execution.proto         # gRPC Agent tool invocation contract
│   ├── schemas/
│   │   ├── events.py                    # Pydantic DTOs (CVFrameDTO, VoiceTranscriptDTO)
│   │   ├── fsm.py                       # FSM State & Event Enum schemas
│   │   ├── context.py                   # MultimodalContextDTO
│   │   └── telemetry.py                 # TelemetryPayloadDTO, AuditLogDTO
│   └── constants/
│       └── zmq_topics.py                # ZeroMQ IPC topic strings & socket paths
├── container-orchestrator/              # Core Control & System Arbiter
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── src/
│   │   ├── main.py                      # Entry point & asyncio event loop initialization
│   │   ├── router/
│   │   │   └── event_router.py          # MultiProtocolEventRouter (ZeroMQ subscriber/puller)
│   │   ├── fsm/
│   │   │   ├── state_machine.py         # TableDrivenFSMEngine
│   │   │   └── states.py                # SystemState enum (IDLE, RUNNING, PAUSED, JAMMED, etc.)
│   │   ├── safety/
│   │   │   └── arbiter.py               # SafetyInvariantArbiter (Guard clauses & invariant matrix)
│   │   ├── context/
│   │   │   └── builder.py               # MultimodalContextBuilder (Sliding window deque)
│   │   ├── agent/
│   │   │   ├── adapter.py               # SLMAgentEngineAdapter (Prompt & action extraction)
│   │   │   └── dispatcher.py            # AgentActionDispatcher
│   │   ├── ipc/
│   │   │   ├── rpmsg_worker.py          # OpenAMPBridgeWorker (Non-blocking /dev/ttyRPMSG0)
│   │   │   └── grpc_clients.py          # gRPC client wrappers
│   │   └── persistence/
│   │       ├── database.py              # WAL-mode SQLite database connection factory
│   │       └── repository.py            # HalfSyncHalfAsyncPersistenceWorker (aiosqlite queue)
├── container-cv/                        # Vision Perception Engine
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
```

```
│       ├── main.py                  # Camera frame loop (V4L2 /dev/video0)
│       ├── detector.py              # YOLOv8 INT8 ONNX/TFLite Inference Engine
│       └── publisher.py             # ZeroMQ Push/Pub frame event publisher
├── container-stt/                   # Speech Recognition Engine
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       ├── main.py                  # ALSA audio stream reader
│       ├── recognizer.py            # Vosk offline speech-to-text pipeline
│       └── publisher.py             # ZeroMQ Pub transcript publisher
├── container-tts/                   # Text-to-Speech Synthesis
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       ├── main.py                  # ZeroMQ Pull subscriber
│       └── synthesizer.py           # Piper TTS speech synthesizer & ALSA audio output
├── container-web/                   # Web HMI Backend & Gateway
│   ├── Dockerfile
│   ├── requirements.txt
│   └── src/
│       ├── main.py                  # FastAPI application entry point
│       ├── api/
│       │   ├── rest_v1.py           # Rest API endpoints for HMI settings & controls
│       │   └── websockets.py        # /ws/telemetry realtime streaming gateway
│       └── cloud/
│           └── egress_worker.py     # Asynchronous AWS API Gateway JSON uploader
└── mcu-firmware/                    # STM32U585 Zephyr RTOS C++ Firmware
    ├── CMakeLists.txt
    ├── prj.conf
    └── src/
        ├── main.cpp                 # Main RTOS task initialization
        ├── rpmsg_driver.cpp         # OpenAMP RPMsg ring buffer ISR & endpoint
        ├── safety_interlock.cpp     # Hardwire E-stop ISR & optical curtain handler
        └── motor_control.cpp        # Closed-loop PID conveyor PWM driver
```

2. Component Tech Stack & Library Map

Component / Container	Role & Responsibilities	Core Libraries & Runtimes	IPC / Protocol Interface
container-orchestrator	Central FSM, Safety Arbiter, Multimodal Context, SLM Agent Engine, OpenAMP Driver	Python 3.12, asyncio, pyzmq, grpcio, pydantic v2, aiosqlite, cobs	ZeroMQ Subscriber, gRPC Server/Client, /dev/ttyRPMSG0
container-cv	Real-time camera ingestion, YOLOv8 INT8 defect/jam detection	Python 3.12, ultralytics, opencv-python-headless, pyzmq, posix_ipc	V4L2 /dev/video0, ZeroMQ PUSH/PUB
container-stt	Far-field offline speech-to-text recognition under 60-90 dB noise	Python 3.12, vosk, sounddevice, numpy, pyzmq	ALSA Audio, ZeroMQ PUB
container-tts	Local speech synthesis for acoustic prompts	Python 3.12, piper-tts, pyzmq, wave	ZeroMQ PULL, ALSA Audio (/dev/snd/*)

Component / Container	Role & Responsibilities	Core Libraries & Runtimes	IPC / Protocol Interface
container-web	FastAPI gateway, WebSocket streaming, AWS cloud egress worker	Python 3.12, fastapi, uvicorn, websockets, httpx, aiosqlite	WebSockets, REST API, AWS HTTPS
mcu-firmware	Deterministic sub-10ms motor PID control, E-stop ISR, optical curtain handler	Zephyr RTOS 3.x, C++17, OpenAMP lib, STM32 HAL	OpenAMP/RPMsg over SRAM ring buffer

3. APIs, Protocols & Interface Contracts

Inter-container and inter-processor communication is strictly governed by compile-time typed schema contracts:

3.1 Protocol Buffer & ZeroMQ Specifications

```
// shared/proto/telemetry.proto
syntax = "proto3";
package eaa.telemetry;

message SensorSnapshot {
  uint64 timestamp_ms = 1;
  float motor_current_amp = 2;
  float infrared_temp_celsius = 3;
  float conveyor_speed_mms = 4;
  bool optical_curtain_clear = 5;
  bool estop_circuit_closed = 6;
}

message SafetyValidationRequest {
  string requested_state = 1;
  string action_intent = 2;
  SensorSnapshot current_sensors = 3;
}

message SafetyValidationResponse {
  bool is_permitted = 1;
  string violation_reason = 2;
  uint32 safety_code = 3;
}

service SafetyArbiterService {
  rpc ValidateTransition (SafetyValidationRequest) returns (SafetyValidationResponse);
}
```

3.2 OpenAMP/RPMsg Hardware Binary Payload (64 Bytes)

```
// mcu-firmware/src/rpmsg_payload.h
#include <stdint.h>

#pragma pack(push, 1)
typedef struct {
    uint8_t  magic_header;      // 0xAA fixed packet start marker
    uint8_t  command_id;        // 0x01: SET_PWM, 0x02: STOP, 0x03: EMERGENCY_HALT
    uint16_t sequence_id;       // Incrementing transaction counter
```

```
    uint32_t timestamp_ms;      // MPU system epoch milliseconds
    int16_t  target_speed_pwm;  // -1000 to +1000 conveyor PWM duty cycle
    uint8_t  safety_flags;      // Bit 0: E-Stop, Bit 1: Curtain, Bit 2: Watchdog
    uint8_t  reserved[51];      // Zero padding to 64 bytes fixed length
    uint16_t crc16_checksum;    // CRC16-CCITT calculation over bytes 0..61
} RPMsgControlFrame;
#pragma pack(pop)
```

4. Low-Level System Design (LLD): Backend Class Specifications

Below are the exact object-oriented class specifications, attribute definitions, method signatures, invariants, and execution patterns for the core backend microservices:

4.1 Class: MultiProtocolEventRouter

• **Module:** container-orchestrator.src.router.event_router
• **Responsibilities:** Asynchronously ingests ZeroMQ perception events (CV defect frames, speech transcripts) and gRPC sensor streams, normalizes them into typed Pydantic DTOs, and dispatches them to the Multimodal Context Builder and FSM Engine.
• **Attributes:**
 - _zmq_context: zmq.asyncio.Context
 - _sub_socket: zmq.asyncio.Socket (ZMQ_SUB / ZMQ_PULL)
 - _context_builder: MultimodalContextBuilder
 - _fsm_engine: TableDrivenFSMEngine
 - _is_running: bool
• **Public Methods:**
 + async start_listening(endpoint: str) -> None
 + async stop_listening() -> None
 + register_handler(topic: str, handler_func: Callable) -> None
 - _normalize_payload(raw_msg: bytes) -> BaseEventDTO

4.2 Class: TableDrivenFSMEngine

• **Module:** container-orchestrator.src.fsm.state_machine
• **Responsibilities:** Enforces deterministic state transitions using an explicit lookup table. Verifies transition structural eligibility before passing state change requests to the Safety Arbiter.
• **Attributes:**
 - _current_state: SystemState (Enum: IDLE, RUNNING, PAUSED, JAMMED, SAFETY_FAULT, RECOVERING)
 - _transition_matrix: Dict[Tuple[SystemState, EventType], SystemState]
 - _safety_arbiter: SafetyInvariantArbiter
 - _lock: asyncio.Lock
• **Public Methods:**
 + get_current_state() -> SystemState
 + can_transition(event: EventType) -> bool
 + async request_transition(event: EventType, context: MultimodalContextDTO) -> Tuple[bool, str]
• **Transition Matrix Rule Table:**
 - (IDLE, START_CMD) -> RUNNING
 - (RUNNING, PAUSE_CMD) -> PAUSED
 - (RUNNING, JAM_DETECTED) -> JAMMED
 - (JAMMED, CLEARANCE_VERIFIED) -> RECOVERING

- (RECOVERING, RESET_CMD) -> IDLE
- (ANY_STATE, HARDWARE_ESTOP) -> SAFETY_FAULT

4.3 Class: SafetyInvariantArbiter

• **Module:** container-orchestrator.src.safety.arbiter
• **Responsibilities:** Evaluates physical hardware invariants prior to command execution. Guarantees that automated voice/vision commands can NEVER override active hardware safety interlocks.
• **Invariants Enforced:**
1. Invariant I-1: Physical E-stop circuit MUST be closed (estop_circuit_closed == True).
2. Invariant I-2: Optical Safety Curtain MUST be clear (optical_curtain_clear == True).
3. Invariant I-3: Motor Current MUST be under thermal ceiling (motor_current_amp <= 2.5A).
4. Invariant I-4: MCU Watchdog heartbeat age MUST be < 100 ms.
• **Public Methods:**
+ validate_safety_invariants(sensors: SensorSnapshotDTO) -> SafetyResultDTO
+ verify_command_intent(intent: AgentIntentDTO, sensors: SensorSnapshotDTO) -> bool

4.4 Class: MultimodalContextBuilder

• **Module:** container-orchestrator.src.context.builder
• **Responsibilities:** Maintains a sliding temporal window of vision frame detections, speech transcripts, and telemetry readings to construct cohesive context for the SLM model engine.
• **Attributes:**
- _vision_buffer: collections.deque[CVFrameDTO] (maxlen=30, TTL=5.0s)
- _voice_buffer: collections.deque[VoiceTranscriptDTO] (maxlen=10, TTL=10.0s)
- _telemetry_buffer: collections.deque[SensorSnapshotDTO] (maxlen=100, TTL=1.0s)
• **Public Methods:**
+ push_vision_frame(frame: CVFrameDTO) -> None
+ push_voice_transcript(transcript: VoiceTranscriptDTO) -> None
+ push_telemetry(snapshot: SensorSnapshotDTO) -> None
+ build_current_context() -> MultimodalContextDTO

4.5 Class: OpenAMPBridgeWorker

• **Module:** container-orchestrator.src.ipc.rpmsg_worker
• **Responsibilities:** Handles low-latency, non-blocking binary frame serialization and deserialization over /dev/ttyRPMSG0 using COBS encoding and CRC16 CCITT validation.
• **Attributes:**
- _device_path: str = '/dev/ttyRPMSG0'
- _fd: int (POSIX non-blocking file descriptor os.O_NONBLOCK)
- _sequence_counter: uint16
• **Public Methods:**
+ async start_bridge_loop() -> None
+ async send_command(cmd_id: int, pwm_speed: int) -> bool
- _calculate_crc16(data: bytes) -> int
- _cobs_encode(payload: bytes) -> bytes

4.6 Class: HalfSyncHalfAsyncPersistenceWorker

• **Module:** container-orchestrator.src.persistence.repository
• **Responsibilities:** Executes non-blocking background queue writes to local SQLite (WAL mode), eliminating disk I/O blocking on time-critical control loops.
• **Attributes:**
- _write_queue: asyncio.Queue[AuditLogDTO]
- _db_connection: aiosqlite.Connection
• **Public Methods:**
+ async enqueue_log(entry: AuditLogDTO) -> None
+ async start_flush_worker() -> None

5. Sequence & Control Flow Blueprint

The complete execution path from perception trigger to physical MCU motor actuation follows a strict 6-stage pipeline:

1. Perception Capture: container-cv detects carton jam via YOLOv8 INT8 model; container-stt transcribes operator voice command ('Clear jam and resume').

2. Event Normalization: ZeroMQ PUSH/PUB sockets publish JSON events to MultiProtocolEventRouter over /tmp/zmq_*.ipc sockets.

3. Context Assembly: MultimodalContextBuilder aggregates frame detections, speech intents, and live telemetry into a validated MultimodalContextDTO.

4. FSM & Safety Verification: TableDrivenFSMEngine verifies transition eligibility (JAMMED -> RECOVERING). SafetyInvariantArbiter executes pre-condition checks against live hardware sensors (confirming E-stop is closed and light curtain is CLEAR).

5. Binary Frame Dispatch: OpenAMPBridgeWorker serializes target PWM speed into a 64-byte RPMsgControlFrame, calculates CRC16, and writes to /dev/ttyRPMSG0 without blocking the asyncio loop.

6. MCU Hardware Execution: STM32U585 MCU receives RPMsg frame in hardware ISR, updates motor PWM timer, and returns telemetry heartbeat in < 10 ms.